CURRENT TRANSFORMER CALCULATION REPORT GENERATOR

OBJECT-ORIENTED ANALYSIS AND DIAGRAMS



ABSTRACT → Django code is used for describing

- THIS DOCUMENT OUTLINES THE CALCULATION AND SELECTION CRITERIA FOR CURRENT TRANSFORMERS (CT) USED IN A HIGH-VOLTAGE SUBSTATION ENVIRONMENT. THE FOCUS IS ON DETERMINING THE KNEE-POINT VOLTAGE (VK) FOR PROTECTION AND DISTANCE RELAY APPLICATIONS. THE SOFTWARE ENSURES ACCURATE CT CALCULATIONS USING KEY PARAMETERS LIKE SHORT-CIRCUIT CURRENT FACTORS, REACTANCE/RESISTANCE RATIOS, AND TRANSFORMER RESISTANCES.

CLASS DIAGRAM (ENTITIES AND METHODS)

- CLASSES:
- - CLIENT
- - CONSULTANT
- - CONTRACTOR/SUBCONTRACTOR
- - CURRENT TRANSFORMER
- - RELAY
- - SYSTEM INFORMATION
- - DOCUMENT

- METHODS: PERFORMCALCULATION(), VALIDATEVK(), GENERATEPDFREPORT(), ETC.

OBJECT-ORIENTED ENTITIES

1. Client
2. Consultant
3. Contractor/Subcontractor
4. Current Transformer (CT)
5. Relay
6. System Information
7. CT Report
8. Project



SOFTWARE PROCESS

1. Initialize project

Consultant/Client/Contractor creates a new project.

Consultant/Client/Contractor adds project details.

- Consultant/Client/Contractor inputs data, calculations Take place.

2. Validate CT Specifications:

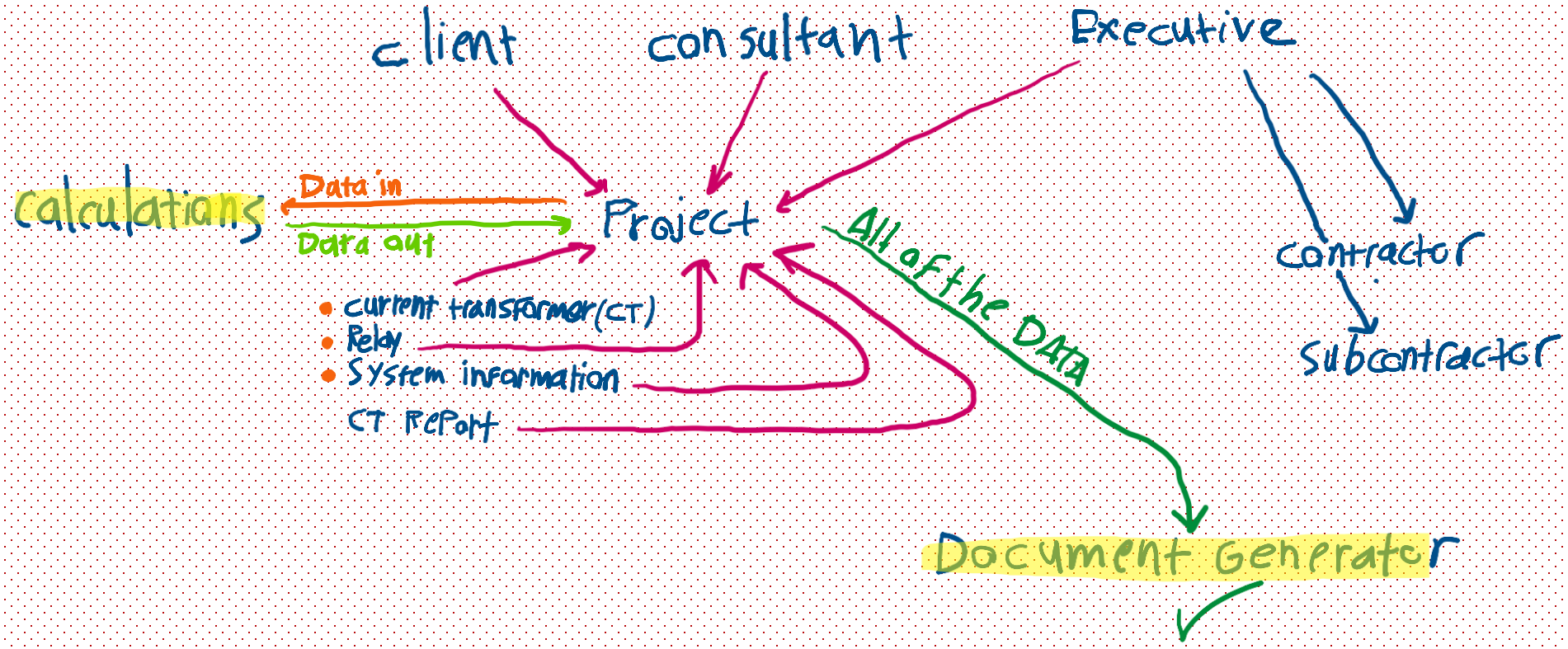
- Consultant/Client/Contractor inputs CT parameters, and the system validates them (According To Calculations.

3. CT Calculation Report Generation:

- All the data is aligned in a proper JSX format

- A PDF report is generated from that JSX.

Abstraction :



In the second slide, according to the <SoftWare Process> slide, we'll begin to perform an analysis on each of these classes:

Class:

- **Attributes . . .**
- **Methods . . .**
- .
- .

But in a diagram form!

Project

Project

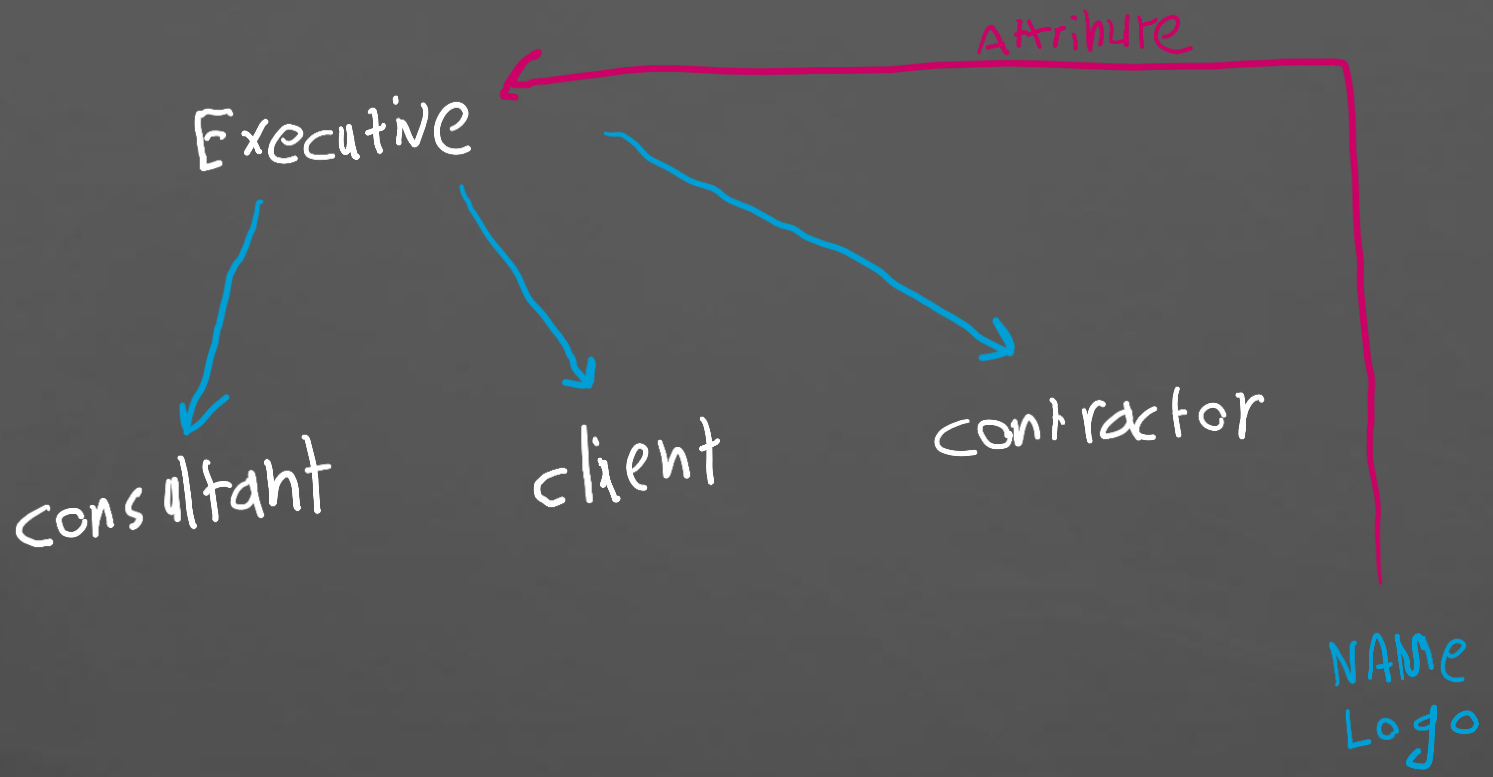
Attributes

Methods

Save/saveas()
Delete()
Generate_output()
Get_CT_Data(ID)
Get_Relay_Data(ID)
CT_New_Add(Data)
Relay_New_add(Data)
Get_system_Info()

client (their own)
consultant (their own)
contractor (their own)
Document title
Revision
DocumentNum = 0
Revision_History = []

Consultant/Client/Contractor



The CT ! (Parent of Relay)

current (CT) transformer

Attributes

Methods

```python
from django.db import models

class CT(models.Model):
    ct_id = models.CharField(max_length=100)
    primary_current = models.FloatField()
    secondary_current = models.FloatField()
    resistance = models.FloatField()
    burden = models.FloatField()
    Vk_required = models.FloatField()
    Vk_calculated = models.FloatField()
    system_info = models.ForeignKey(SystemInformation, on_delete=models.CASCADE)

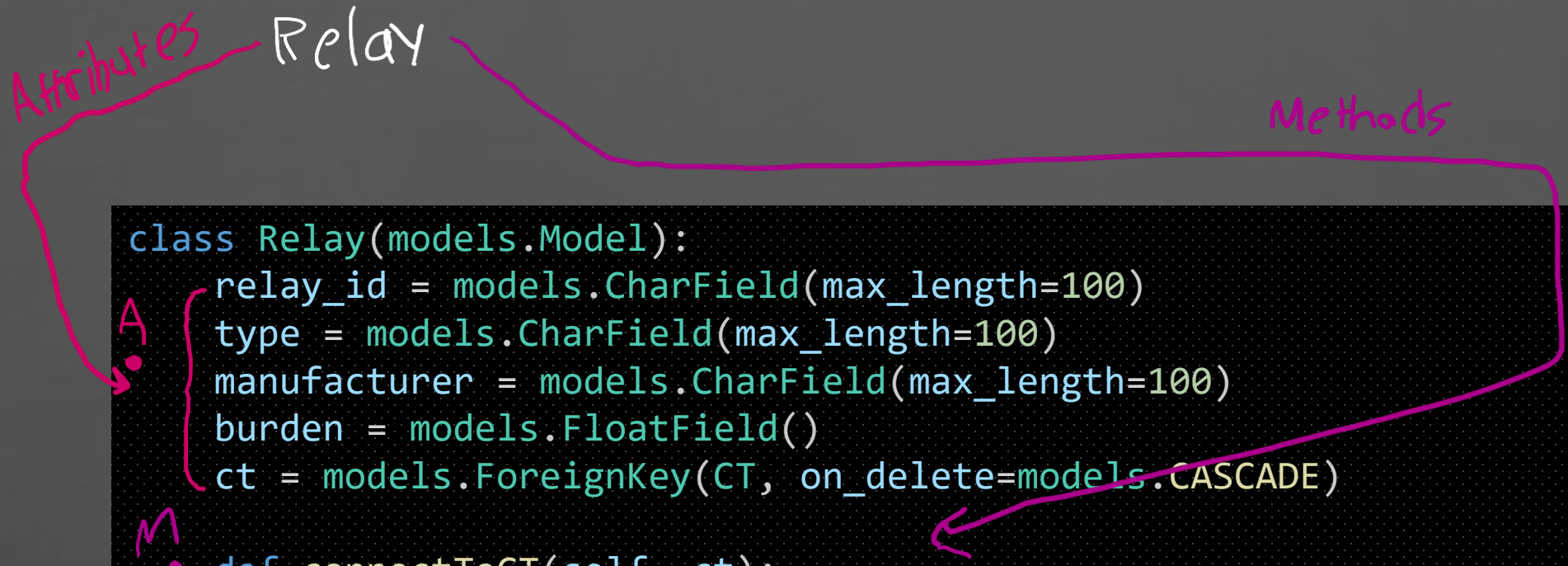

    def acceptRelay(self, relay):
        # Check if the relay is a fit
        if relay.burden <= self.burden:
            return True
        return False

    def calculateVk(self):
        # Logic to calculate Vk based on CT properties
        pass

    def validateVk(self):
        # Validation of calculated Vk against required Vk
        return self.Vk_calculated >= self.Vk_required
```

Accept logic

The Relay! (child of CT)




```
class Relay(models.Model):
    relay_id = models.CharField(max_length=100)
    type = models.CharField(max_length=100)
    manufacturer = models.CharField(max_length=100)
    burden = models.FloatField()
    ct = models.ForeignKey(CT, on_delete=models.CASCADE)

    def connectToCT(self, ct):
        if ct.acceptRelay(self):
            self.ct = ct
            self.save()
            return True
        return False
```